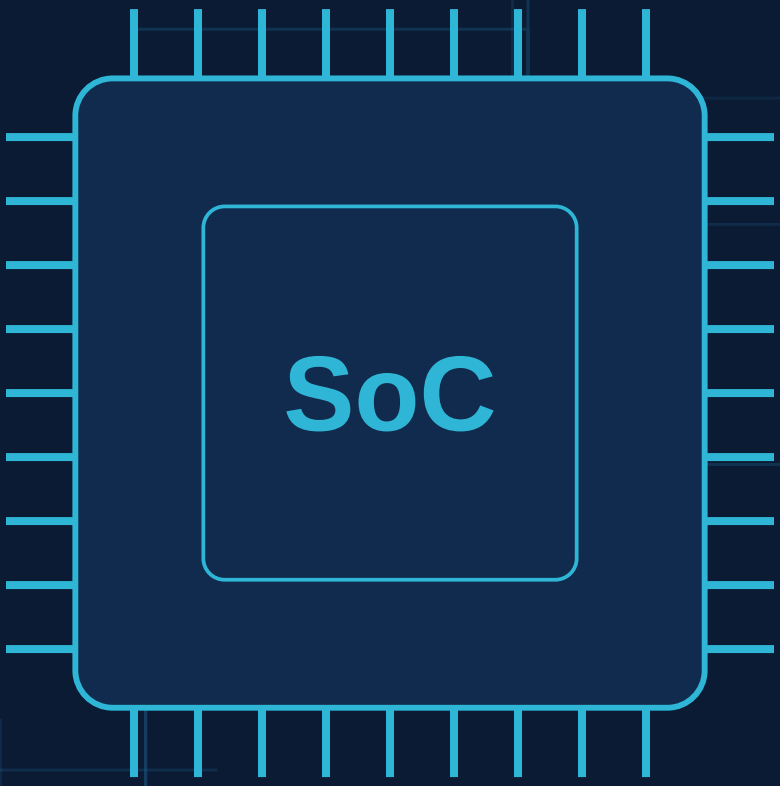


Final System

Low-Power Multi-Clock Communication System

RTL Design • Verification • Digital Backend Implementation

PROJECT REPORT

Ziad Abdelhafeez

Table of Contents

1. System Overview **3**
1.1 What the system does 3
1.2 Block-by-block functionality 4
1.3 Backend constraints summary 4
2. Functional Verification **5**
2.1 Test 1 — RF_Wr_CMD (0xAA) 5
2.2 Test 2 — RF_Rd_CMD (0xBB) 6
2.3 Test 3 — ALU_OP_W_OP_CMD (0xCC) 7
2.4 Test 4 — ALU_OP_W_NOP_CMD (0xDD) 8
2.5 Simulation Transcript 9
3. Digital Backend Implementation Flow **10**
3.1 SpyGlass Lint & CDC 10
3.2 Synthesis 10
3.3 Formality — Post-Synthesis 11
3.4 DFT Insertion 11
3.5 Formality — Post-DFT 11
4. Conclusion **11**

Table of Figures

Figure 1: Final System top-level block diagram. 3
Figure 2: Register-file write: write 0x55 to addr 0x4. 5
Figure 3: Register-file read-back of addr 0x4, confirming the Test 1 write. 6
Figure 4: ALU add with fresh operands: 10 + 20. 7
Figure 5: ALU AND, reusing the operands already staged by Test 3. 8
Figure 6: QuestaSim transcript for the four-command regression. 9

1. System Overview

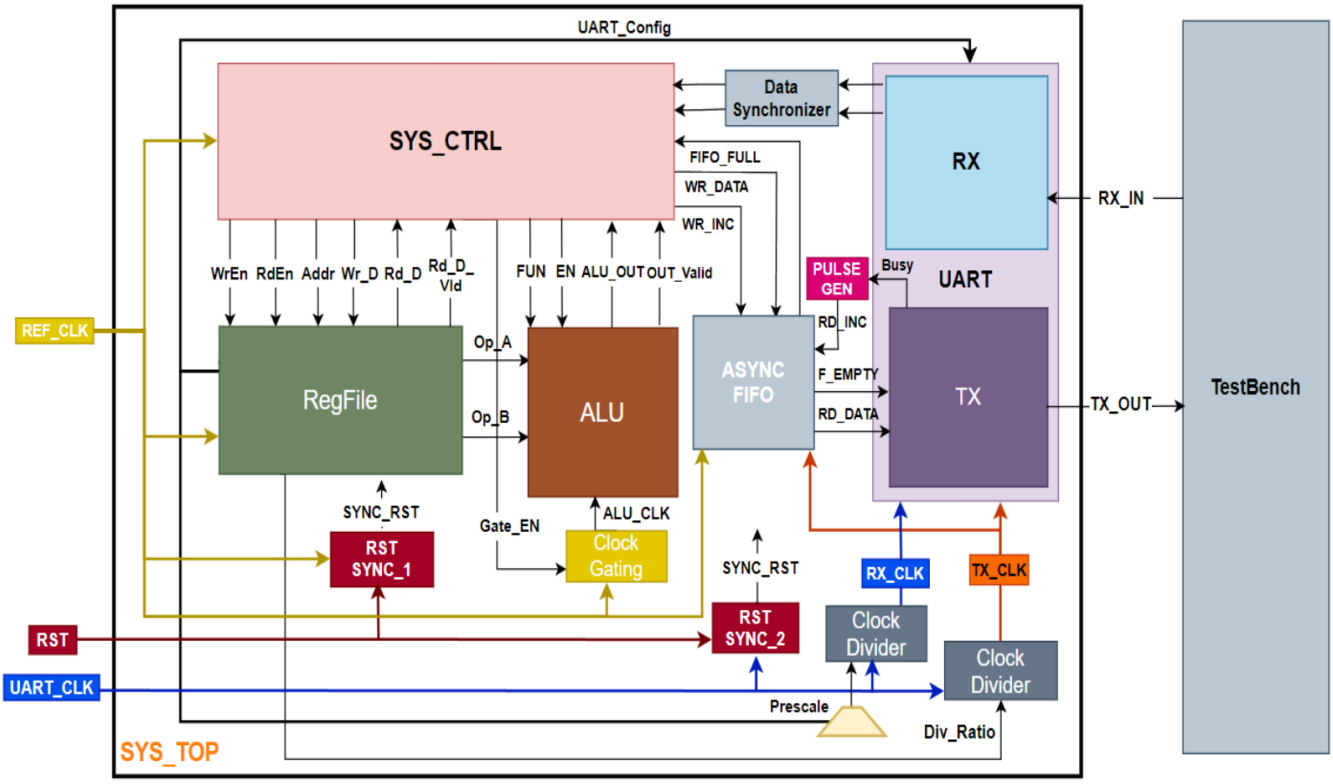


Figure 1: Final System top-level block diagram.

1.1 What the system does

The Final System is a small UART-controlled SoC: a remote master sends command frames over UART_RX, the system decodes and executes them using a register file and ALU, and streams the result back out over UART_TX. It runs across two independent clock domains — a 50 MHz REF_CLK for the control/compute path and a 3.686 MHz UART_CLK for the serial front-end — bridged by dedicated reset synchronizers, a data synchronizer, and an asynchronous FIFO.

1.2 Block-by-block functionality

- **SYS_CTRL** — The system's control unit. Decodes the four UART command opcodes, drives RegFile and ALU to execute them, and streams results back through the FIFO. Designed and verified from scratch during this project as a 13-state FSM with Mealy-style, same-cycle byte handling.
- **RegFile** — 8×16 register file. Holds general-purpose data plus two reserved slots (ALU operand A/B) and two configuration registers (UART parity/prescale, clock-divider ratio).
- **ALU** — Executes 12 operations — add, subtract, multiply, divide, the six logic ops, compare, and two shifts — on two operands, producing a 16-bit result with a one-cycle-aligned valid pulse.
- **Clock Gating** — Gates REF_CLK into the ALU on demand via an integrated clock-gating cell, cutting dynamic power while the ALU is idle.
- **Clock Dividers (×2)** — Derive UART_RX_CLK and UART_TX_CLK from UART_CLK at software-configurable ratios (register-controlled prescale / division).
- **UART RX / TX** — The serial front-end. Frames and deframes bytes with configurable parity, driven by the baud/prescale settings loaded into RegFile.
- **PULSE_GEN** — Converts UART_TX's level-style Busy signal into a single-cycle pulse that drives the FIFO's read-pointer increment.
- **RST_SYNC (×2)** — Two-flop reset synchronizers, one per clock domain, turning the asynchronous chip reset into a clean, domain-safe release.
- **DATA_SYNC** — Synchronizes a received byte and its valid pulse from the UART_RX clock domain into REF_CLK atomically, so SYS_CTRL always sees a byte and its valid flag update together.
- **ASYNC_FIFO** — Gray-coded, pointer-synchronized dual-clock FIFO carrying result bytes from SYS_CTRL (REF_CLK) across to UART_TX (UART_TX_CLK).

1.3 Backend constraints summary

The synthesis constraint set (cons.tcl) defines two master clocks — REF_CLK at 50 MHz and UART_CLK at 3.686 MHz — and three generated clocks derived from them: ALU_CLK (REF_CLK, ÷1), RX_CLK (UART_CLK, ÷1), and TX_CLK (UART_CLK, ÷32). Every clock carries 0.2 ns setup / 0.1 ns hold uncertainty, 0.05 ns transition on the masters, and a dont-touch on the full clock network. REF_CLK/ALU_CLK and the UART_CLK family are declared as mutually asynchronous clock groups. Every non-clock, non-reset port carries an input or output delay of 20% of its related clock period, inputs are driven by a buffer cell, and outputs are loaded at 0.1 pF. Timing is signed off across both the slow (ss) and fast (ff) operating corners. The same treatment — dedicated clock, its own async group, matching delay/driving-cell/load constraints — was extended to the scan interface (SI, SE, SO) for the DFT stage.

2. Functional Verification

A UART-level testbench acts as the external master: it bit-bangs real UART frames onto UART_RX_IN and passively decodes whatever comes back on UART_TX_O, exercising the four supported command types end to end through the real RTL (UART framing, clock-domain crossings, FIFO, and the SYS_CTRL FSM included) rather than driving SYS_CTRL's internal ports directly.

2.1 Test 1 — RF_Wr_CMD (0xAA)

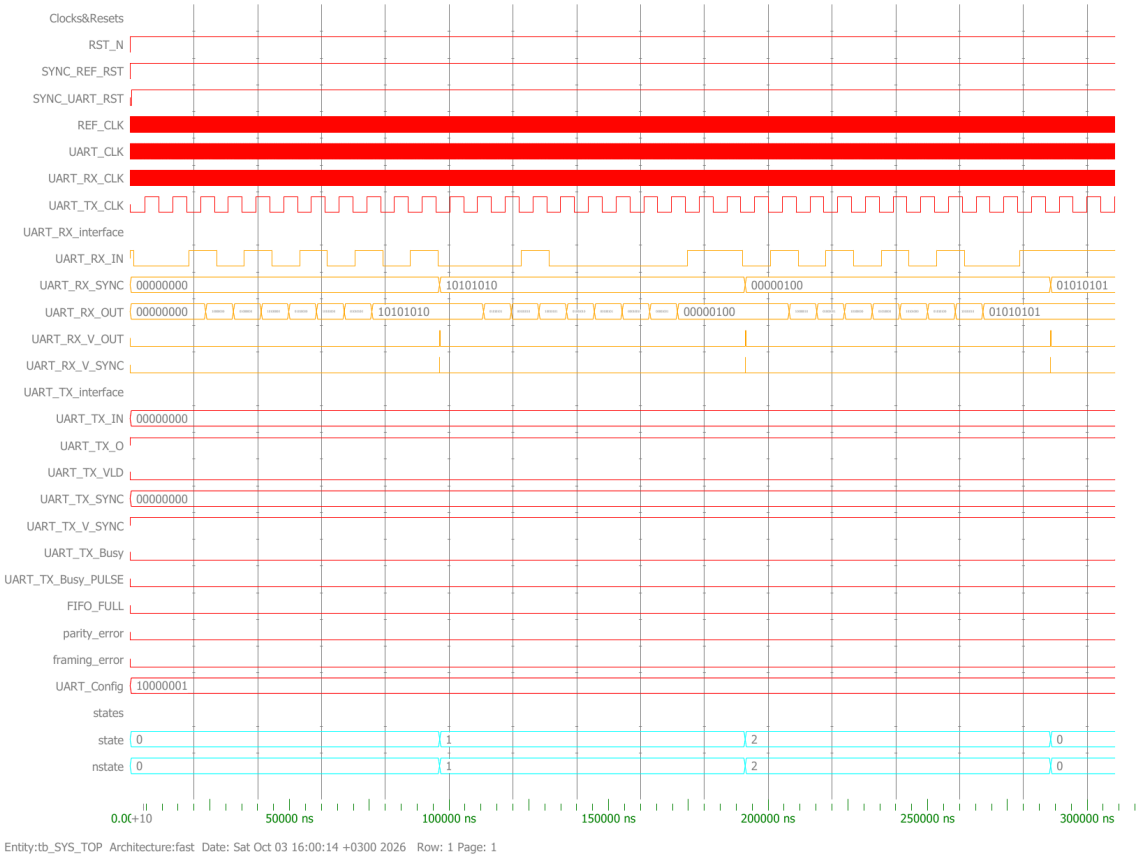


Figure 2: Register-file write: write 0x55 to addr 0x4.

The master sends 0xAA, address 0x4, then data 0x55. SYS_CTRL stages the address and fires a single-cycle RF_WrEn pulse into RegFile. No reply is expected for a write, and none is sent — confirmed by the clean WrEn pulse with zero UART_TX activity in this window.

2.2 Test 2 — RF_Rd_CMD (0xBB)

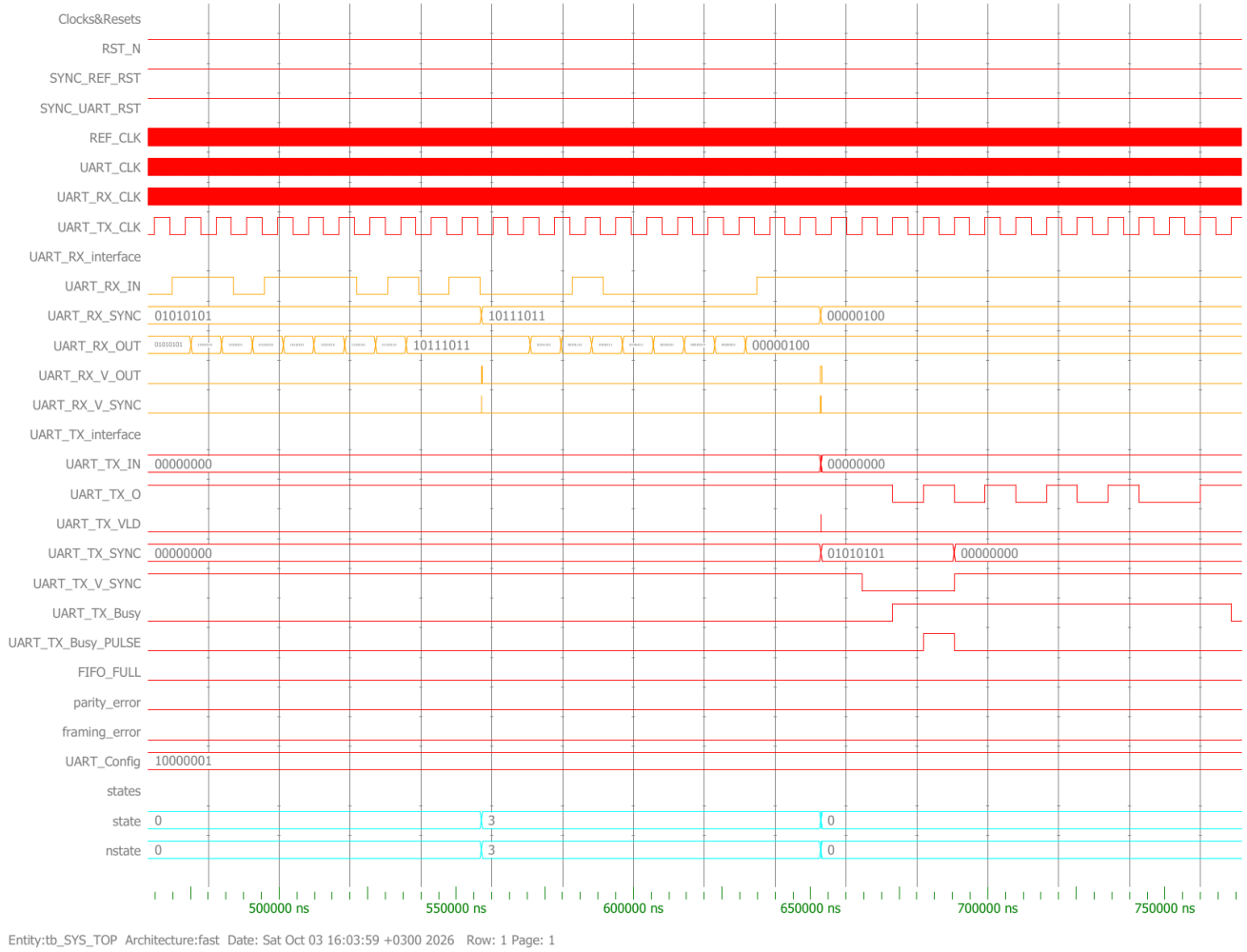


Figure 3: Register-file read-back of addr 0x4, confirming the Test 1 write.

The master sends 0xBB, address 0x4. SYS_CTRL pulses RdEn, waits one cycle for RdData_Valid, and streams the result out. UART_TX_SYNC shows 0x55 — exactly the byte written in Test 1 — proving the write landed and the read path works end to end.

2.3 Test 3 — ALU_OP_W_OP_CMD (0xCC)

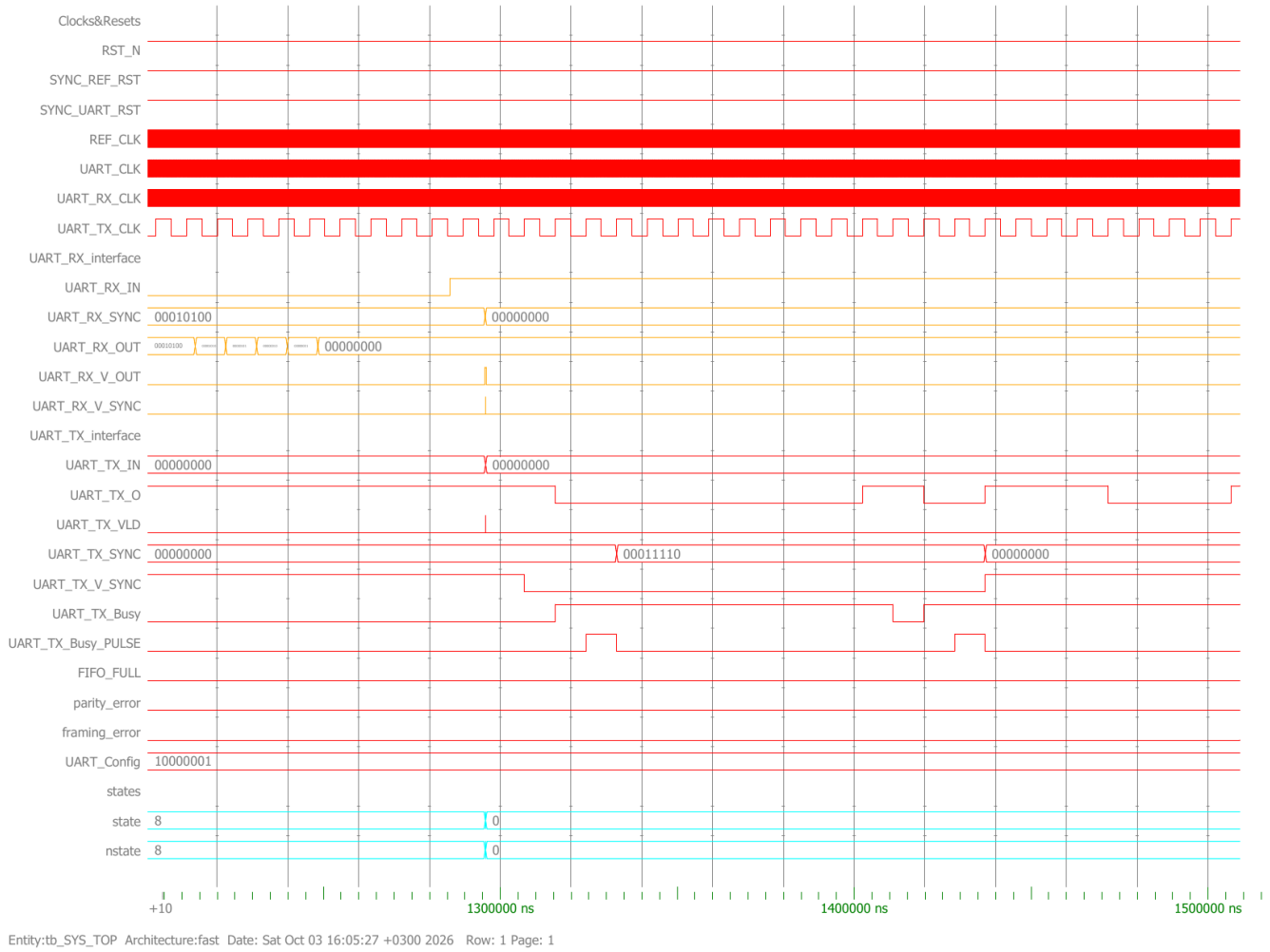


Figure 4: ALU add with fresh operands: 10 + 20.

The master sends 0xCC with operands 10 and 20 and function code ADD. SYS_CTRL stages both operands into REG0/REG1, runs the ALU, and streams the 16-bit result out as two bytes. UART_TX_SYNC shows 0x00 then 0x1E (=30) — the addition is correct, and the high-byte-first framing for 16-bit ALU results works as designed.

2.4 Test 4 — ALU_OP_W_NOP_CMD (0xDD)

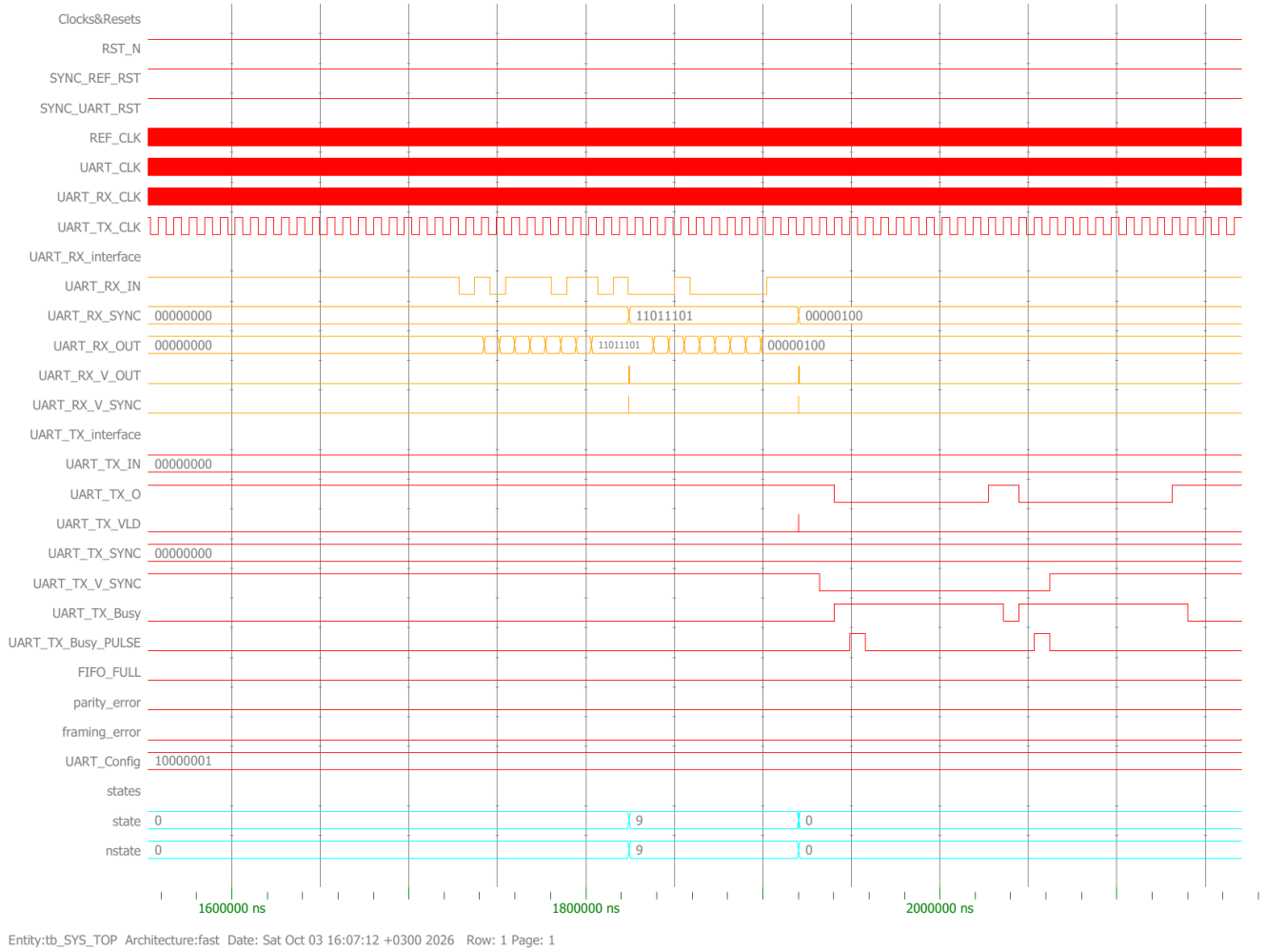


Figure 5: ALU AND, reusing the operands already staged by Test 3.

The master sends only 0xDD plus the function code (AND) — no operand bytes. SYS_CTRL runs the ALU directly on whatever is already sitting in REG0/REG1 (10 and 20 from Test 3). Result returned is 0x00/0x00 (10 AND 20 = 0), confirming operand reuse works without a fresh write.

2.5 Simulation Transcript

The console transcript below confirms all four commands were sent and every expected reply byte was captured on UART_TX_O, matching the waveform evidence above.

```
# === TEST 1: RF_Wr_CMD - write 0x55 to addr 0x4 ===
# [96576000] MASTER sent byte 0xaa
# [192062000] MASTER sent byte 0x04
# [287548000] MASTER sent byte 0x55
#
# === TEST 2: RF_Rd_CMD - read addr 0x4 (expect 0x55) ===
# [556646000] MASTER sent byte 0xbb
# [652132000] MASTER sent byte 0x04
# [755617000] >>> UART_TX_O byte #1 = 0x55
#
# === TEST 3: ALU_OP_W_OP_CMD - 10 + 20 (expect hi=0x00 lo=0x1E) ===
# [1008034000] MASTER sent byte 0xcc
# [1103521000] MASTER sent byte 0x0a
# [1199007000] MASTER sent byte 0x14
# [1294493000] MASTER sent byte 0x00
# [1397979000] >>> UART_TX_O byte #2 = 0x00
# [1502146000] >>> UART_TX_O byte #3 = 0x1e
#
# === TEST 4: ALU_OP_W_NOP_CMD - 10 AND 20 (expect hi=0x00 lo=0x00) ===
# [1824007000] MASTER sent byte 0xdd
# [1919493000] MASTER sent byte 0x04
# [2022981000] >>> UART_TX_O byte #4 = 0x00
# [2127148000] >>> UART_TX_O byte #5 = 0x00
```

Figure 6: QuestaSim transcript for the four-command regression.

3. Digital Backend Implementation Flow

With functional verification complete, the design was carried through the full digital backend flow: static CDC/lint signoff, synthesis, formal equivalence checking, DFT (scan) insertion, and a second formal equivalence check against the scan-inserted netlist.

3.1 SpyGlass Lint & CDC

A full CDC analysis was run on SYS_TOP against a dedicated SGDC (clock/reset definitions, port domain abstractions, and Gray-pointer/quasi-static declarations). Four convergence warnings came back from the first pass, and each was traced back to the actual RTL rather than blanket-waived:

- **FIFO empty/full flags** — standard Gray-coded-pointer reconvergence, the textbook async-FIFO pattern. Resolved by declaring the pointer buses with *cdc_attribute ... -type gray* so SpyGlass verifies the Gray property directly instead of flagging a generic bus crossing.
- **SYS_CTRL's shared FSM state register** — a structural false positive: the tool's static fan-in sees every synchronized signal that ever touches the state register as "converging" on every state-derived output, even though the FSM's states are mutually exclusive at runtime. Documented via *cdc_attribute ... -type mutually_exclusive*.
- **DATA_SYNC's byte + valid-pulse pair** — safe by construction: the synchronized data byte and its qualifying valid pulse update atomically on the same clock edge inside DATA_SYNC, so combining them in SYS_CTRL's opcode decode is correct usage, not an unsynchronized multi-bit crossing. Declared via *cdc_attribute ... -type separate_enable*.

Static configuration registers (UART_Config, DIV_RATIO) were declared quasi-static, since they are written once at start-up and held for the rest of operation. With these attributes in place, the CDC rule set passed cleanly with no unresolved convergence or synchronization violations.

3.2 Synthesis

Synthesized against the constraint set described in Section 1.3 (two master clocks, three generated clocks, full uncertainty/transition/dont-touch coverage, I/O delay and driving-cell/load constraints on every functional port, and slow/fast corner signoff). The design compiled cleanly with no combinational loops or inferred latches, and setup/hold timing closed with no violating paths across both corners.

3.3 Formality — Post-Synthesis

Formal equivalence checking between the RTL and the synthesized gate-level netlist completed successfully, with no failing or inconclusive points — confirming synthesis preserved the verified functional behavior with no logical changes introduced by optimization or mapping.

3.4 DFT Insertion

SYS_TOP_dft.v adds the scan port set (scan_clk, scan_rst, test_mode, SE, SI, SO) and the clock/reset mux structure needed to route scan_clk and scan_rst into both functional clock domains during test mode. cons.tcl was extended with a dedicated SCAN_CLK definition, its own asynchronous clock group, and matching delay/driving-cell/load constraints on SI/SE/SO, plus a set_case_analysis forcing test_mode active for scan-mode timing analysis.

The first insert_dft attempt failed outright (TEST-211, with TEST-374 flagging missing clock information for every sequential cell). Root-caused to three set_dft_signal declarations (scan_clk's ScanClock type, scan_rst's Reset type, and test_mode's Constant type) incorrectly using *-view existing_dft* instead of *-view spec* — a view meant for re-describing an already-scan-inserted netlist, not for specifying new test structure on fresh RTL. With all three corrected to *-view spec*, create_test_protocol correctly picked up the scan clock, and scan chains were built and routed successfully with dft_drc, preview_dft, and insert_dft all completing cleanly and scan coverage exceeding the 98% target.

3.5 Formality — Post-DFT

A second formal equivalence check — between the pre-DFT and scan-inserted netlists — completed successfully with no failing points, confirming that scan insertion preserved the design's original functional behavior in mission mode.

4. Conclusion

This project took the Final System from a single missing block — the control unit — through a complete, working chip. SYS_CTRL was designed as a compact FSM, debugged down to real RTL bugs (an under-declared internal bus width in DATA_SYNC that was silently truncating every received byte to one bit, found directly from waveform evidence), and verified end to end with a UART-level testbench exercising all four supported commands over real serial framing.

From there the design was carried through the full digital backend: a CDC/lint signoff where every reported warning was individually traced to its root cause in the RTL rather than waived on faith, a clean synthesis against a fully-specified constraint set, formal equivalence confirming synthesis changed nothing functionally, DFT insertion (including recovering from a real tool-flow bug in the DFT signal view declarations), and a second formal check confirming scan insertion was equally transparent.

The result is a UART-controlled ALU/register-file SoC that is functionally verified, CDC-clean, timing-closed, formally equivalent at every stage, and test-ready — taken from RTL all the way to a scan-inserted, backend-signed-off netlist.